PartnerAI - Project Technical Specification

PartnerAI Technical Blueprint & Specification

Date: April 15, 2026

Status: Confidential / Intellectual Property (IP) Profile

Project Objective: To develop a highly integrated, autonomous personal mentor system that leverages local LLMs and vector memory to optimize human productivity and habit formation.

---

1. Executive Summary

PartnerAI is a next-generation "AI Life Partner" designed to bridge the gap between static task management and dynamic mentorship. Unlike generic LLM interfaces, PartnerAI proactively extracts personal facts, manages a relational knowledge graph (Smart Blocks), and adapts its interaction strategy based on Reinforcement Learning from Human Feedback (RLHF).

2. System Architecture

The platform is built on a hybrid local-first architecture to ensure data privacy and low-latency interaction.

- **Backend:** Flask (Python) as the central orchestration hub.
- **Intelligence Layer:** `llama.cpp` serving lightweight models (e.g., Phi-3) locally.
- **Memory Layer:** Dual-infrastructure:
- **Relational (SQLite):** For structured data (Users, Habits, Focus Sessions, Team Sync).
- **Vector (FAISS):** For unstructured context (Personal facts, Domain Knowledge).
- **Frontend:** Responsive HTML5/Vanila CSS dashboard with real-time SSE (Server-Sent Events) for AI streaming.

3. Core Innovations (Patentable Logic)

3.1 Adaptive Strategic RLHF (Strategy Selector)

The system employs a `StrategySelector` class that differentiates between interaction styles (e.g., `direct_action`, `motivational_push`, `strategic_analysis`).

- **Mechanism:** Epsilon-greedy selection based on historical feedback scores.
- **Innovation:** The AI dynamically alters its "personality" and prompt instructions based on real-time success metrics, effectively "learning" how to best mentor individual users.

3.2 Heuristic Personal Fact Extraction

A background "Memory Engine" monitors every conversation interaction.

- **Mechanism:** Uses a heuristic filter (`maybe_extract_memory`) to detect self-disclosures (e.g., "I work as a developer", "I struggle with morning focus").
- **Innovation:** Seamless, friction-less memory building without requiring manual user input, enabling long-term personalization.

3.3 Autonomous Task Synthesis

The `_extract_tasks_from_chat` mechanism utilizes a secondary LLM pass to analyze mentor advice.

- **Mechanism:** Parses the Mentor's streaming response in a background thread to extract concrete, actionable JSON tasks.
- **Innovation:** Closes the loop between "advice" and "action" by automatically updating the user's dashboard with tasks generated during conversation.

3.4 Relational "Smart Blocks"

A hierarchical and networked knowledge system (`smart_blocks.py`).

- **Mechanism:** Enables linking "Idea", "Task", and "Learning" blocks into a semantic graph.
- **Innovation:** Provides a unified interface for project management and knowledge capture within a single AI-mentored ecosystem.

4. Technical Data Schema

The `partnerai.db` manages 15+ specialized tables, including:

- `users`: Core profile and state tracking.
- `ai_user_memory`: Key-value memories with confidence scores.
- `focus_sessions`: Granular productivity metrics (Focus scores, feedback).
- `group_tasks`: Multi-agent/Collaborative task distribution.
- `knowledge_blocks`: Relational storage for the Smart Block system.

5. User Growth Lifecycle

1. Onboarding: AI-driven roadmap generation based on goals and schedule.
2. Growth Loop: Focus sessions Feedback AI Strategy Adaptation Task Refinement.

3. Collaboration: Squad-based mentorship and group milestones.

NOTE:

This document is optimized for conversion to PDF via standard Markdown-to-PDF exporters. All logic described is implemented in the existing codebase across `app.py`, `rag_engine.py`, `local_llm.py`, and `rlhf/strategy_selector.py`.